

TECHNICAL REPORT

IoT / EMBEDDED SYSTEMS

FreeRTOS Queue-Based Sensor Data Transfer -
Architectural & Implementation
Engineering Document

Daily Technical Report, Day 35

Project	Queue-Based Producer-Consumer Communication Between ESP32 Tasks
Target MCU	ESP32 Development Board running ESP-IDF / FreeRTOS
Input Signals	Potentiometer ADC reading + digital IR state
Output Signals	PWM LED brightness + digital LED state
Inter-Task Transport	FreeRTOS Queue using sensor_message_t copies
Queue Experiment	Depth 1 first, then depth 5; producer intentionally faster than consumer
Execution Status	Hardware operational; queue pipeline verified; timing/overflow test in progress
Prepared By	Malak Alsharqawi
Prepared For	IOT Training Program

Contents

1	Daily Overview
1.1	Summary
1.2	Goal & Submission Scope
1.3	Tasks Completed
2	Conceptual Foundation: Why the Queue Replaces Shared Globals
2.1	Message Passing Instead of Shared Sensor State
2.2	Exact Task 3.6 Race Condition
2.3	Why a Global Struct Alone Would Not Solve It
2.4	Queue Message Passing and Data Ownership
2.5	What the Queue Contains Under the Hood
2.6	Blocking vs. Polling
2.7	Full and Empty Queue Symmetry
2.8	Queue Depth, Throughput, Backpressure, and Data Loss
2.9	Stale Queue Data vs. Stale Shared-Variable Data
2.10	Queue vs. Mutex vs. Semaphore
3	System Architecture & Concurrency Model
3.1	Producer-Consumer Ownership Model
3.2	Scheduler Blocking Model
4	Hardware Test Setup & Schematic
5	Queue Design and API Configuration
5.1	sensor_message_t and QueueHandle_t
5.2	xQueueCreate()
5.3	xQueueSend()
5.4	xQueueReceive()
6	Implementation Breakdown
6.1	Sensor Task
6.2	Output Task
6.3	app_main() Initialization Order
7	Queue Depth Experiment & Timing Analysis
7.1	Depth 1
7.2	Depth 5
7.3	Stale Data and Latency
8	Troubleshooting Log
9	Verification & Requirements Compliance
10	Definition of Done
11	Next Steps
12	Appendix: Core Implementation Snippets

1. Daily Overview

1.1 Summary

This task upgrades the previous two-task ESP32 design from unsynchronized shared global variables to explicit FreeRTOS message passing. The Sensor Task reads the potentiometer and digital IR input, packages both values into one `sensor_message_t` structure, and sends a complete copy through a FreeRTOS Queue. The Output Task blocks on the queue, receives its own copy, and then updates PWM brightness and the digital LED output.

The core engineering objective is not only to make the hardware work, but to prove that the new architecture solves the specific Task 3.6 race condition: the consumer must no longer observe an ADC value from one sampling moment and an IR value from another. The queue turns the two values into one atomic application-level message and also provides synchronization with the FreeRTOS scheduler.

1.2 Goal & Submission Scope

Goal: Replace the Task 3.6 shared sensor globals with a proper FreeRTOS Queue and experimentally verify queue-full behavior at two queue depths.

- Use the same Sensor Task and Output Task roles from the previous design.
- Define one message structure containing potentiometer value and IR state.
- Pass all sensor data through `xQueueCreate()`, `xQueueSend()`, and `xQueueReceive()`.
- Sensor Task sends on a fixed interval; Output Task blocks on `xQueueReceive()` instead of polling.
- Test queue depth 1 first, then depth 5, while the producer runs faster than the consumer.
- Log every send and receive using `ESP_LOG` and use the logs as experimental evidence.

1.3 Tasks Completed

- Removed direct shared `adc_raw` and `ir_state` communication from the task interface.
- Defined `sensor_message_t` to carry a coherent sensor snapshot.
- Created a `QueueHandle_t` and initialized the queue before task creation.
- Moved `xQueueSend()` inside the Sensor Task loop so every sensor sample can actually be transmitted.
- Configured the Output Task to block on `xQueueReceive()` and update PWM / LED outputs only after a valid message is received.
- Validated the hardware path after the queue integration; a small visible latency remains when deliberate processing delays are enabled.

2. Conceptual Foundation: Why the Queue Replaces Shared Globals

The previous implementation stored `adc_raw` and `ir_state` as global variables. The Sensor Task wrote them independently, while the Output Task read them independently. Because FreeRTOS may context-switch between normal C statements, the consumer could observe a logically inconsistent pair even if each individual integer access was valid.

```
// Previous shared-memory pattern
int adc_raw;
int ir_state;

// Sensor Task
adc_raw = new_adc;
ir_state = new_ir;

// Output Task
use(adc_raw, ir_state);
```

Race condition: A context switch can occur after `adc_raw` is updated but before `ir_state` is updated. The Output Task can therefore read a new ADC value together with an old IR value.

2.1 Message Passing Instead of Shared Sensor State

```
typedef struct {
    int adc_raw;
    int ir_state;
} sensor_message_t;
```

The Sensor Task now creates a complete local snapshot before sending. FreeRTOS copies that fixed-size structure into queue-managed storage. The Output Task later receives another copy into its own local variable. This separates ownership cleanly: producer local memory -> queue storage -> consumer local memory.

Design Property	Shared Globals	FreeRTOS Queue
Data ownership	Both tasks touch the same variables	Each task owns its local copy
Snapshot consistency	Can mix values from different moments	ADC and IR travel together as one message
Waiting strategy	Usually polling or timed reads	Consumer can block efficiently
Buffering	No history; only current variables	Temporary FIFO buffer with defined depth
Overflow behavior	Not explicit	Controlled by queue depth and send timeout

2.2 The Exact Task 3.6 Race Condition - An Inconsistent Snapshot

The original problem was not simply that two integers were global. The real concurrency problem was that `adc_raw` and `ir_state` represented one logical sensor sample but were updated as two separate memory operations. FreeRTOS is allowed to schedule another ready task between those statements. The Output Task could therefore read a new ADC value together with an old IR value and treat that mixed pair as if it came from one sampling instant.

Sensor Task	Output Task
-----	-----
<code>adc_raw = 2500</code>	
context switch	
+----->	<code>reads adc_raw = 2500</code>
	<code>reads ir_state = 0 <- old sample</code>
<-----	<code>context switch</code>
<code>ir_state = 1</code>	
Intended sample: {2500, 1}	
Observed sample: {2500, 0}	

Important distinction: a single integer read/write may be atomic on the ESP32, but that does not make a multi-field sensor sample atomic as a whole. The Queue solves the logical snapshot problem by transferring the fields together.

2.3 Why a Global Struct Alone Would Not Solve It

Packing the two globals into one struct improves organization, but it does not automatically add synchronization. If that struct remains global and the Sensor Task updates its members one after another, a context switch can still occur between the member assignments. The Output Task can still observe a partially updated structure.

```
// Better organization, but still shared mutable memory
typedef struct {
    int adc_raw;
    int ir_state;
} sensor_message_t;

sensor_message_t shared_data;

shared_data.adc_raw = 2500;
/* scheduler may switch tasks here */
shared_data.ir_state = 1;
```

This is why the design change is not merely 'put the values in a struct'. The real change is from shared mutable state to message passing: the producer first builds a complete local struct and only then sends one complete item.

2.4 Queue Message Passing and Data Ownership

With the Queue design, each stage owns a different copy of the message. The Sensor Task owns `sensor_data` while it is sampling. `xQueueSend()` copies that item into FreeRTOS-managed Queue storage. Later, `xQueueReceive()` copies the oldest queued item into `received_data` owned by the Output Task. The two tasks no longer need to read and write the same sensor payload in RAM.

Sensor Task local memory
+-----+
sensor_data
adc_raw = 2500
ir_state = 1
+-----+
xQueueSend() copies sizeof(sensor_message_t)
v
+-----+
FreeRTOS Queue storage
{2500, 1}
+-----+

```

      | xQueueReceive() copies and removes oldest item
      v
+-----+
| Output Task local memory |
| received_data = {2500, 1} |
+-----+

```

A successful send therefore does not mean the Output Task has already processed the message. It only means that a copy has entered the Queue. Sampling, enqueueing, waiting, receiving, and processing are separate stages.

2.5 What the Queue Contains Under the Hood

Conceptually, a FreeRTOS Queue is more than an array. It combines fixed-size storage with metadata and scheduler integration. FreeRTOS must know the item size, maximum depth, number of messages currently waiting, read/write positions, and which tasks are blocked waiting to send or receive.

```

+-----+
| Queue control information |
+-----+
| item size = sizeof(sensor_message_t) |
| maximum length = QUEUE_DEPTH |
| messages currently waiting |
| read position / write position |
| tasks waiting because queue is empty |
| tasks waiting because queue is full |
+-----+
| Fixed-size message storage (circular style) |
+-----+

```

The storage behaves conceptually like a circular buffer. When an item is received, FreeRTOS advances its read position and frees one slot; when an item is sent, it advances the write position. Items do not need to be shifted through memory after every receive.

2.6 Blocking vs. Polling - Why the Queue Also Synchronizes Tasks

The Queue is not only a data container; it is also a synchronization point. With polling, the Output Task would wake repeatedly just to ask whether new data exists. With a blocking receive, FreeRTOS can remove the Output Task from CPU scheduling until either a message arrives or its timeout expires.

```

Polling approach:
RUNNING -> check -> no data -> delay/check -> no data -> check ...

Blocking Queue approach:
RUNNING -> xQueueReceive()
      |
      +-- queue has data -> receive immediately
      |
      +-- queue empty -> BLOCKED
                          |
                          producer sends message
                          |
                          v
                          READY
                          |
                          scheduler selects task
                          |
                          v
                          RUNNING

```

A blocked task does not burn CPU cycles waiting. This is different from `vTaskDelay()`: a delay sleeps for a fixed time even if data arrives early, while `xQueueReceive()` can wake as soon as the event it is waiting for occurs.

2.7 Full and Empty Queue Symmetry

The send and receive sides have a useful symmetry. A receiver may block because the Queue is empty; a sender may block because the Queue is full. Sending an item can wake a waiting receiver, while receiving an item can free a slot and wake a waiting sender.

```

QUEUE EMPTY
|
receiver calls xQueueReceive(wait > 0)
|
receiver -> BLOCKED
|
producer sends
|
receiver -> READY

QUEUE FULL
|
sender calls xQueueSend(wait > 0)
|
sender -> BLOCKED
|
consumer receives
|
sender -> READY

```

In this assignment, a zero-wait send is especially useful for the overflow experiment because it makes queue-full behavior explicit: the send fails immediately, the existing queued item remains, and the new reading is not stored.

2.8 Queue Depth, Throughput, Backpressure, and Data Loss

Queue depth controls how much temporary mismatch between producer and consumer rates can be absorbed. Depth 1 can hold only one waiting message. Depth 5 can absorb a larger burst, but neither depth changes how fast the Output Task actually processes data. If the producer remains permanently faster, every finite Queue will eventually fill.

- Zero-wait send on a full Queue: the new message is rejected immediately; the old queued message is not overwritten.
- Nonzero send timeout on a full Queue: the Sensor Task may enter the Blocked state while waiting for space.
- Blocking a producer creates backpressure: the slow consumer indirectly slows future production.
- A larger Queue reduces immediate overflow during bursts but consumes more RAM and may increase latency.

2.9 Stale Queue Data vs. Stale Shared-Variable Data

The word stale means something different after moving to a Queue. A queued item can be perfectly valid and internally consistent but old relative to the current physical input because newer samples are waiting behind it. That is Queue latency. In the old global-variable design, stale data could instead mean that the consumer reread an old value without knowing whether a new sample had arrived, and it could also combine values from different sampling moments.

```

Current potentiometer position -> 3000

Queue backlog:
Front
  v
[1200] [1600] [2100] [2600] [3000]
                                ^
                                newest

xQueueReceive() returns 1200 first.
1200 is valid, but stale relative to the current input.

```

This is why a deeper Queue is not automatically 'better'. If every sample matters, buffering history may be correct. If only the latest state matters, carrying a long backlog can make the physical response feel slow.

2.10 Queue vs. Mutex vs. Semaphore

FreeRTOS provides several synchronization primitives, but they solve different problems. This task uses a Queue because data itself must move from one task to another.

Primitive	Primary purpose	What it would mean in this task
Queue	Transfer messages + synchronize	Send ADC/IR snapshot from Sensor Task to Output Task
Mutex	Protect shared resource	Would protect shared memory, but would not itself transport the sample
Semaphore	Signal event / resource count	Could signal that something happened, but carries little/no payload

Mental model: Sensor Task creates one complete snapshot -> xQueueSend() copies it into Queue storage -> Output Task blocks until data exists -> xQueueReceive() copies and removes the oldest item -> Output Task owns and processes that copy.

3. System Architecture & Concurrency Model

The firmware is organized as a simple producer-consumer pipeline. Hardware acquisition and hardware actuation remain in separate tasks; the FreeRTOS Queue is the only sensor-data path between them.

```
[ Potentiometer + IR Input ]
    |
    v
[ Sensor Task ]
local sensor_message_t
    |
    xQueueSend()
    |
    v
[ sensor_queue ]
FIFO + blocking
    |
    xQueueReceive()
    |
    v
[ Output Task ]
local received_data copy
    |
    v
[ PWM LED + LED ]
```

3.1 Producer-Consumer Ownership Model

- Sensor Task is the producer: it acquires ADC and IR values and creates one complete message.
- sensor_queue is the communication and synchronization object managed by FreeRTOS.
- Output Task is the consumer: it receives a complete message, updates PWM duty, and sets the digital LED state.

3.2 Scheduler Blocking Model

If the queue is empty, xQueueReceive() can move the Output Task to the Blocked state. A blocked task does not continuously consume CPU time. When the Sensor Task sends a message, FreeRTOS can move the receiver back to Ready and schedule it according to task priority.

```
Queue empty -> Receiver may BLOCK
Send succeeds -> Waiting receiver becomes READY
Queue full -> Sender may BLOCK or FAIL, depending on xTicksToWait
Receive succeeds -> One slot becomes free
```

4. Hardware Test Setup & Schematic

The provided CirqIt Designer schematic documents the bench wiring used to verify the queue-controlled firmware. It shows the ESP32 development board, potentiometer input wiring, breadboard power distribution, and red/green LED output connections. The firmware additionally reads a digital IR state; a discrete IR sensor module is not visibly represented in this supplied schematic, so this figure is treated as the provided bench wiring reference rather than a complete sensor inventory.

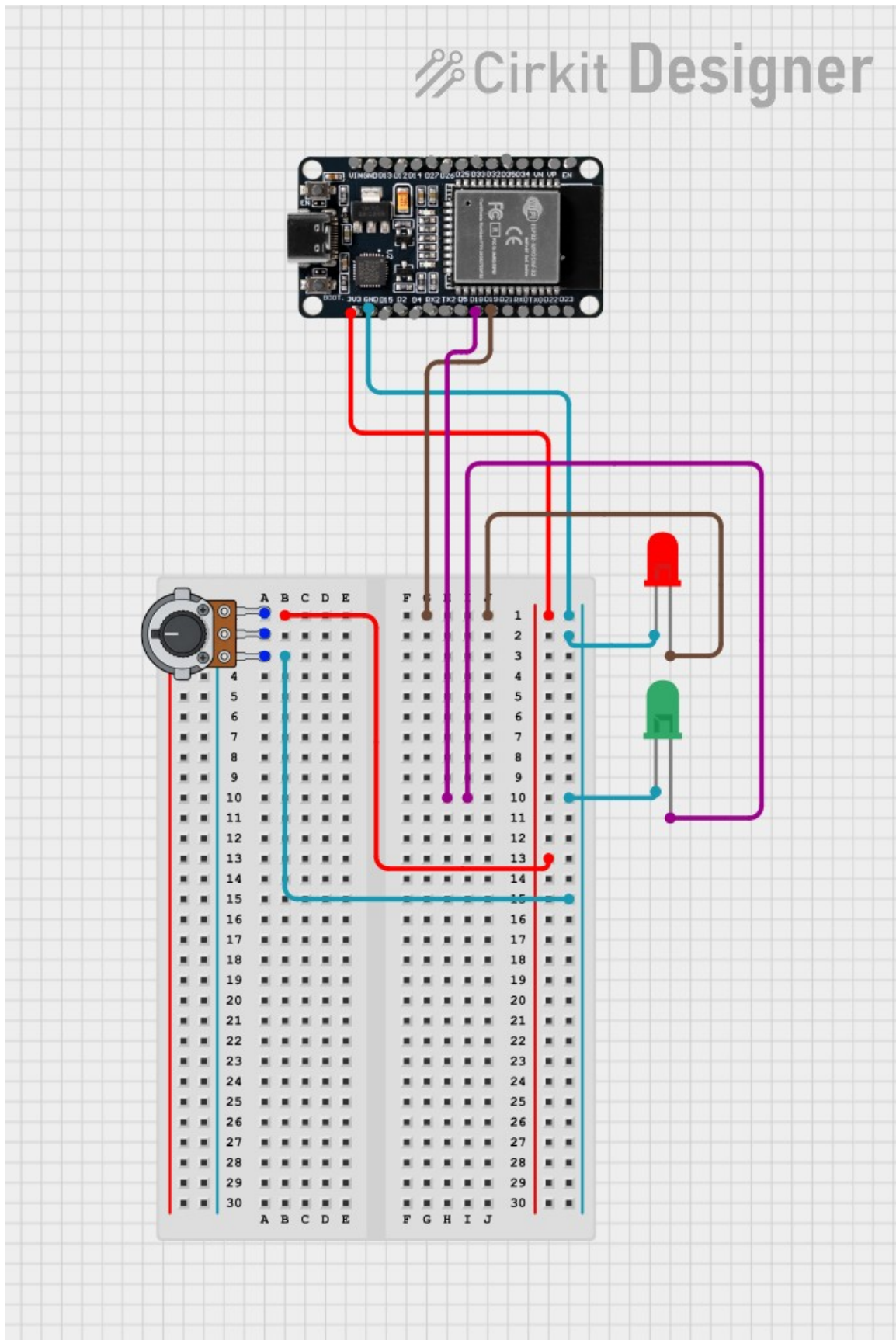


Figure 1. Provided CirkIt Designer schematic for the ESP32 queue-task hardware test setup.

Hardware Element	Role in Firmware
ESP32	Runs ESP-IDF / FreeRTOS, ADC acquisition, queue logic, PWM, and GPIO control
Potentiometer	Analog input used to generate adc_raw for PWM control
Red / Green LEDs	Visual output indicators driven by PWM and digital GPIO logic
Breadboard / jumpers	Power and signal interconnect for the bench test
IR digital state	Read by firmware and included in sensor_message_t; sensor module is not shown in the provided schematic

5. Queue Design and API Configuration

5.1 sensor_message_t and QueueHandle_t

```
typedef struct {
    int adc_raw;
    int ir_state;
} sensor_message_t;

static QueueHandle_t sensor_queue;
```

sensor_message_t defines the application-level protocol between the two tasks. QueueHandle_t is an opaque reference to the FreeRTOS queue object; it is not the sensor data itself.

5.2 xQueueCreate()

```
sensor_queue = xQueueCreate(
    QUEUE_DEPTH,
    sizeof(sensor_message_t)
);
```

The first argument is the maximum number of messages that may wait simultaneously. The second argument is the fixed byte size of every queue item. The queue is created before the tasks so neither task can access an uninitialized handle.

5.3 xQueueSend()

```
if (xQueueSend(sensor_queue, &sensor_data, 0) == pdPASS) {
    ESP_LOGI(SENSOR_TAG, "Sensor data sent");
} else {
    ESP_LOGW(SENSOR_TAG, "Queue full - send failed");
}
```

xQueueSend() copies sizeof(sensor_message_t) bytes from &sensor_data into queue storage. With zero wait, a full queue causes immediate failure and the new reading is not stored. This makes overflow behavior easy to observe in the experiment.

5.4 xQueueReceive()

```
if (xQueueReceive(sensor_queue, &received_data, portMAX_DELAY) == pdTRUE) {
    // received_data now contains the oldest queued message
}
```

xQueueReceive() copies the oldest FIFO message into received_data and removes that item from the queue. portMAX_DELAY makes the consumer block efficiently while the queue is empty.

6. Implementation Breakdown

6.1 Sensor Task - Producer

```
void sensor_task(void *pvParameters)
{
    sensor_message_t sensor_data;

    while (1)
    {
        adc_oneshot_read(adc1_handle, POT_CHANNEL, &sensor_data.adc_raw);
        sensor_data.ir_state = gpio_get_level(IR_GPIO);

        xQueueSend(sensor_queue, &sensor_data, 0);
        vTaskDelay(pdMS_TO_TICKS(SENSOR_INTERVAL_MS));
    }
}
```

The critical implementation rule is that `xQueueSend()` must be inside the infinite loop. In an earlier version it was placed after `while(1)`, which made the send code unreachable. The Sensor Task therefore read hardware continuously but never placed a message in the queue.

6.2 Output Task - Consumer

```
void output_task(void *pvParameters)
{
    sensor_message_t received_data;

    while (1)
    {
        if (xQueueReceive(sensor_queue, &received_data, portMAX_DELAY) == pdTRUE)
        {
            ledc_set_duty(LEDCLOW_SPEED_MODE, LEDC_CHANNEL_0,
                          received_data.adc_raw);
            ledc_update_duty(LEDCLOW_SPEED_MODE, LEDC_CHANNEL_0);
            gpio_set_level(BLINK_GPIO, !received_data.ir_state);
        }
    }
}
```

The Output Task uses only `received_data`. It no longer reads producer-owned sensor globals, so the specific Task 3.6 shared-data race is removed.

6.3 app_main() Initialization Order

```
setup();
sensor_queue = xQueueCreate(QueueDepth, sizeof(sensor_message_t));
if (sensor_queue == NULL) return;
xTaskCreate(sensor_task, "Sensor Task", 2048, NULL, 1, NULL);
xTaskCreate(output_task, "Output Task", 2560, NULL, 2, NULL);
```

7. Queue Depth Experiment & Timing Analysis

The experiment deliberately configures the Sensor Task to produce messages faster than the Output Task consumes them. This exposes overflow and latency behavior instead of hiding it behind a perfectly balanced timing configuration.

7.1 Depth 1

```
QUEUE_DEPTH = 1
Sensor interval = 100 ms
Output processing time = 500 ms
Send wait = 0 ticks
```

```
0 ms    SEND A -> success
~0 ms   RECEIVE A -> success; consumer begins processing
100 ms  SEND B -> success (one queue slot available)
200 ms  SEND C -> fail: queue still contains B
300 ms  SEND D -> fail
400 ms  SEND E -> fail
500 ms  consumer ready -> RECEIVE B
```

Depth 1 provides almost no burst buffering. It produces low queueing latency for messages that are accepted, but new readings are easily dropped when the consumer is deliberately slowed.

7.2 Depth 5

Changing only QUEUE_DEPTH from 1 to 5 allows several readings to accumulate while the consumer is busy. Fewer immediate send failures are expected, but the consumer may process older readings before reaching the newest value.

Metric	Depth 1	Depth 5
Burst capacity	1 waiting message	5 waiting messages
Overflow	Occurs quickly	Delayed
Dropped readings	More likely with zero-wait send	Less likely during short bursts
Latency / backlog	Usually smaller	Can grow as queue fills
Best observation	Clear overflow behavior	Buffering vs stale-data trade-off

7.3 Stale Data and Latency

A deeper queue does not make the consumer faster. If the producer remains faster for long enough, any finite queue eventually fills. A queued message may also become stale: it can be internally correct and coherent, yet old relative to the current physical potentiometer position.

8. Troubleshooting Log

Issue	Root Cause	Resolution
xTaskCreate incompatible pointer type	String tag such as SENSOR_TASK was passed as the task function argument	Pass sensor_task / output_task as the first argument; keep text strings only for task names or log tags
Hardware outputs did not update after queue integration	xQueueSend() was placed after while(1), making the send path unreachable	Move xQueueSend() inside the Sensor Task loop after both sensor values are acquired
Output Task appeared frozen	xQueueReceive(..., portMAX_DELAY) correctly blocked forever because no producer message was ever sent	Once xQueueSend() ran inside the loop, the receiver woke and hardware updated
Hardware response felt slightly slow	Deliberate output processing delay and/or queue backlog introduced visible latency	Keep the delay for overflow testing; remove or reduce it for normal responsive operation
Potential overflow ambiguity	Nonzero send timeout can block the producer and hide immediate overflow	Use xQueueSend(..., 0) for the depth-comparison experiment to make full-queue failures explicit
Key debugging lesson: With RTOS code, a function can compile correctly and still never execute. Control-flow placement inside or outside an infinite task loop is as important as the API call itself.		

9. Verification & Requirements Compliance

The hardware is now operating with the queue-based task pipeline. The user observed a small amount of slowness, which is consistent with the intentionally slow consumer configuration used to create queue pressure. A serial monitor log was not supplied with this report, so exact counts of successful sends, failed sends, and receive timestamps are not claimed here.

Requirement	How It Is Satisfied	Status
Same Sensor and Output task roles	Existing two-task architecture retained	Met
Struct message containing ADC + IR	sensor_message_t contains adc_raw and ir_state	Met
xQueueCreate / xQueueSend / xQueueReceive	All three APIs are integrated in the design	Met
No shared sensor globals	Output consumes received_data rather than direct sensor globals	Met in queue path
Output blocks instead of polling	xQueueReceive(..., portMAX_DELAY) blocks on empty queue	Met
Depth 1 and depth 5 comparison	QUEUE_DEPTH is designed to be changed between the two experimental runs	Ready / requires captured logs for final evidence
ESP_LOG on send and receive	Logging statements included in producer and consumer paths	Met in implementation
Serial Monitor evidence	Exact log capture not included in the material supplied for this report	Pending capture

10. Definition of Done

Criterion	Status
Global adc_raw / ir_state communication removed from the task data path	Met
Queue created before task execution	Met
Sensor Task sends one coherent message per sample cycle	Met
Output Task blocks on queue and updates hardware only after receive	Met
Depth 1 overflow behavior captured in log	Pending final Serial Monitor capture
Depth 5 comparison captured with same timing	Pending final Serial Monitor capture
Technical report explains race-condition change and stale-data trade-off	Met

11. Next Steps

- Run the depth-1 configuration with Sensor Task faster than Output Task and capture at least 10-20 seconds of Serial Monitor logs.
- Change only QUEUE_DEPTH to 5 and repeat the test with identical producer and consumer timing.
- Count or visually compare SEND OK, SEND FAILED, and RECEIVE OK events between the two runs.
- Document whether the larger queue reduced drops but increased visible latency or backlog.
- After the experiment, remove the deliberate Output Task processing delay for normal responsive hardware operation.
- Optionally add uxQueueMessagesWaiting(sensor_queue) to diagnostic logs to show queue occupancy during the experiment.
- Commit the final source code, Serial Monitor evidence, schematic, and report to the training repository.

11.1 Expected Final Engineering Conclusion

The FreeRTOS Queue solves the specific Task 3.6 race condition because the Sensor Task and Output Task no longer directly exchange data through independently updated global sensor variables. Instead, the producer creates one coherent snapshot, FreeRTOS copies it into synchronized queue storage, and the consumer later receives its own copy. Queue depth does not eliminate throughput mismatch: depth 1 exposes overflow quickly, while depth 5 buffers a larger burst at the cost of additional backlog and potential stale-data latency.

12. Appendix: Core Implementation Snippets

12.1 Queue Definitions

```
typedef struct
{
    int adc_raw;
    int ir_state;
} sensor_message_t;

static QueueHandle_t sensor_queue;
```

12.2 Producer Send Path

```
sensor_message_t sensor_data;

while (1)
{
    ret = adc_oneshot_read(adc1_handle, POT_CHANNEL,
                          &sensor_data.adc_raw);
    sensor_data.ir_state = gpio_get_level(IR_GPIO);

    if (xQueueSend(sensor_queue, &sensor_data, 0) != pdPASS)
        ESP_LOGW(SENSOR_TAG, "Queue full - send failed");
    else
        ESP_LOGI(SENSOR_TAG, "SEND OK: adc_raw=%d, ir_state=%d",
                  sensor_data.adc_raw, sensor_data.ir_state);

    vTaskDelay(pdMS_TO_TICKS(SENSOR_INTERVAL_MS));
}
```

12.3 Consumer Receive Path

```
sensor_message_t received_data;

while (1)
{
    if (xQueueReceive(sensor_queue, &received_data, portMAX_DELAY) == pdTRUE)
    {
        ledc_set_duty(LEDCLOW_SPEED_MODE, LEDC_CHANNEL_0,
                      received_data.adc_raw);
        ledc_update_duty(LEDCLOW_SPEED_MODE, LEDC_CHANNEL_0);
        gpio_set_level(BLINK_GPIO, !received_data.ir_state);
    }
}
```

12.4 Queue Initialization

```
sensor_queue = xQueueCreate(QueueDepth, sizeof(sensor_message_t));
if (sensor_queue == NULL)
{
    ESP_LOGE("MAIN", "Failed to create sensor queue");
    return;
}

xTaskCreate(sensor_task, "Sensor Task", 2048, NULL, 1, NULL);
xTaskCreate(output_task, "Output Task", 2560, NULL, 2, NULL);
```

End of Technical Report